App Educativo com Tags NFC

https://gemini.google.com/app/452e2b5ffa762fd5

User prompt: Gere tudo acima e preciso de um passo a passo como irei fazer em sal as instruções

Response: Para executar esse Rali de Banco de Dados de 100 minutos no laboratório do IFS Campus Tobias Barreto, unificamos a dinâmica de sala, o banco de dados (SQLite), o backend (Flask) e o frontend (Flutter).

Como Conduzir a Aula (Instruções para o Professor)

1. **Preparação Pré-Aula (Bastidores)**

   Antes dos 100 minutos

   1. **Gravação das Tags:** Use o app *NFC Tools* (Android/iOS) para gravar as NTAG213. Adicione um registro do tipo "Texto" em 10 tags com IDs de `tag_01` a `tag_10`.
   2. **Banco de Dados:** Rode o script SQLite (abaixo) para popular as missões e as equipes.
   3. **Ambiente Físico:** Cole as 10 tags no laboratório (ex: monitores, bancadas laterais, porta, sua mesa).

2. **Briefing e Divisão**

   Minutos 0 a 15

   Divida a turma em 10 grupos. Cada grupo escolhe 1 aluno para usar o celular (Scanner) com o seu app Flutter aberto. No telão do laboratório, projete a página web do painel Flask (que exibirá o ranking atualizando automaticamente). Entregue a regra clara: **"Vocês escaneiam a tag, resolvem o SQL no PC, e digitam apenas a RESPOSTA CURTA no app."**

3. **A Largada (O Sprint de Código)**

   Minutos 15 a 85

   Inicie a competição. O aluno-scanner levanta, bate o celular em uma tag e volta para a equipe com a missão (ex: *"Qual o ID do produto mais caro?"*). Eles criam a query no SQLite, executam e digitam a resposta (ex: *"42"*) no app. Você, como Professor EBTT, circula prestando mentoria apenas para destravar raciocínios (sem dar a resposta pronta).

4. **Hard Stop e Premiação**

   Minutos 85 a 90

   Encerre a recepção de respostas na API (você pode fazer isso via banco de dados ou simplesmente avisando "Mãos para cima"). O telão mostrará a equipe vencedora baseada no maior XP. Faça uma salva de palmas e entregue uma pontuação bônus na avaliação da disciplina.

5. **Debriefing**

   Minutos 90 a 100

   Identifique no painel quais desafios tiveram mais erros (ex: um `GROUP BY` que gerou confusão). Projete esse desafio específico no telão e resolva junto com a turma, consolidando o aprendizado prático antes do sinal bater.

1. Banco de Dados MVP (SQLite)

Este é o esquema enxuto para o Rali. Salve como `banco_rali.sql` e execute no seu ambiente local.

```
-- Tabelas
CREATE TABLE equipes (
    id INTEGER PRIMARY KEY,
    nome TEXT NOT NULL,
    xp_total INTEGER DEFAULT 0
);

CREATE TABLE missoes (
    id INTEGER PRIMARY KEY,
    tag_id TEXT UNIQUE NOT NULL, -- O texto que está gravado na NTAG213 (ex: 'tag_01')
    pergunta TEXT NOT NULL,      -- O que aparece no app
    resposta_curta TEXT NOT NULL,-- O gabarito exato (Hash/Output)
    xp_valor INTEGER NOT NULL
);

-- Dados Iniciais (Popule as equipes e os desafios)
INSERT INTO equipes (nome, xp_total) VALUES ('Equipe A', 0), ('Equipe B', 0);

-- Desafios SQL (Modelo Resposta Curta)
INSERT INTO missoes (tag_id, pergunta, resposta_curta, xp_valor) VALUES
('tag_01', 'DDL: Qual comando apaga a estrutura de uma tabela inteira?', 'DROP TABLE', 10),
('tag_02', 'DQL: Rode um SELECT COUNT(*) na tabela Funcionarios. Qual o número total?', '145', 20),
('tag_03', 'DQL: Faça um JOIN entre Vendas e Produtos. Qual o nome do produto mais vendido?', 'Notebook De
```

```
('tag_04', 'DML: Qual cláusula é obrigatória usar com UPDATE para não alterar todas as linhas?', 'WHERE',
('tag_05', 'DQL (Boss): Use subquery para achar o aluno com a maior nota. Digite o nome dele.', 'Juliane',
```

2. Backend API (Python + Flask)

Este script recebe a leitura, valida a resposta e atualiza o XP. Pode rodar direto no PC do laboratório.

```
from flask import Flask, request, jsonify
import sqlite3

app = Flask(__name__)

def get_db_connection():
    conn = sqlite3.connect('rali.db')
    conn.row_factory = sqlite3.Row
    return conn

@app.route('/api/validar', methods=['POST'])
def validar_missao():
    dados = request.json
    id_equipe = dados.get('id_equipe')
    tag_id = dados.get('tag_id')
    resposta_aluno = str(dados.get('resposta')).strip().lower()

    conn = get_db_connection()
    missao = conn.execute('SELECT * FROM missoes WHERE tag_id = ?', (tag_id,)).fetchone()

    if not missao:
        return jsonify({"erro": "Tag não encontrada!"}), 404

    # Validação da resposta (ignorando maiúsculas/minúsculas)
    if missao['resposta_curta'].lower() == resposta_aluno:
        # Acertou! Soma o XP
        conn.execute('UPDATE equipes SET xp_total = xp_total + ? WHERE id = ?',
                     (missao['xp_valor'], id_equipe))
        conn.commit()
        conn.close()
        return jsonify({"status": "sucesso", "mensagem": f"Correto! +{missao['xp_valor']} XP"}), 200
    else:
        conn.close()
        return jsonify({"status": "erro", "mensagem": "Resposta incorreta. Tente novamente!"}), 400

if __name__ == '__main__':
    # Roda na rede local do laboratório do IFS
    app.run(host='0.0.0.0', port=5000)
```

3. Frontend App (Flutter)

A lógica principal no Flutter usa o `nfc_manager` para ler a NTAG213 e a biblioteca `http` para bater na sua API Flask.

```
import 'package:flutter/material.dart';
import 'package:nfc_manager/nfc_manager.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

class ScannerScreen extends StatefulWidget {
  @override
  _ScannerScreenState createState() => _ScannerScreenState();
}

class _ScannerScreenState extends State<ScannerScreen> {
  String _tagLida = "Aproxime da etiqueta...";
  TextEditingController _respostaController = TextEditingController();

  void _iniciarLeituraNFC() async {
    bool isAvailable = await NfcManager.instance.isAvailable();
    if (!isAvailable) return;

    NfcManager.instance.startSession(onDiscovered: (NfcTag tag) async {
      // Extrai o texto da NTAG213 (Depende da formatação NDEF)
      var ndef = Ndef.from(tag);
      if (ndef != null && ndef.cachedMessage != null) {
        String payload = String.fromCharCodes(
            ndef.cachedMessage!.records.first.payload.skip(3) // Pula o header de idioma
        );

        setState(() {
          _tagLida = payload; // Ex: 'tag_03'
        });
```

```
      }
      NfcManager.instance.stopSession();
    });
  }

  void _enviarResposta() async {
    // Substitua pelo IP da máquina do laboratório onde o Flask está rodando
    var url = Uri.parse('http://192.168.1.100:5000/api/validar');

    var response = await http.post(url,
      headers: {"Content-Type": "application/json"},
      body: jsonEncode({
        "id_equipe": 1, // ID da equipe logada
        "tag_id": _tagLida,
        "resposta": _respostaController.text
      }),
    );

    var jsonResponse = jsonDecode(response.body);
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(content: Text(jsonResponse['mensagem']))
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Rali SQL - IFS')),
      body: Padding(
        padding: EdgeInsets.all(16.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            ElevatedButton(
              onPressed: _iniciarLeituraNFC,
              child: Text('ESCANEAR NOVA MISSÃO'),
            ),
            SizedBox(height: 20),
            Text("Missão atual: $_tagLida", style: TextStyle(fontSize: 18)),
            TextField(
              controller: _respostaController,
              decoration: InputDecoration(labelText: "Digite o resultado da Query (Curto)"),
            ),
            SizedBox(height: 20),
            ElevatedButton(
              onPressed: _enviarResposta,
              child: Text('ENVIAR RESPOSTA E GANHAR XP'),
            )
          ],
        ),
      ),
    );
  }
}
```

Essa arquitetura pode ser conectada ao *FlowEdu* ou ao *Tech Stop* no futuro, mas este formato isolado garante que você tenha um MVP robusto rodando perfeitamente nos seus 100 minutos de aula.

User prompt: Quero criar do zero e colocar no GitHub Pages e usar i google sheets é possível?

Response: Sim, é perfeitamente possível e é uma das arquiteturas mais inteligentes e rápidas para validar um MVP educacional. Você não precisará pagar hospedagem e terá um sistema "Serverless" (sem servidor).

Nesse modelo, o **GitHub Pages** hospeda o aplicativo web (HTML/JS) e o **Google Sheets** atua como seu banco de dados. Para conectar os dois, usamos o **Google Apps Script**, que transforma sua planilha em uma API (recebendo requisições da sua página no GitHub).

⚠️ O Grande Alerta Técnico: Web NFC e iPhones

Como você vai rodar isso direto no navegador (via GitHub Pages) em vez de um app nativo instalado, usaremos a **Web NFC API**.

- **Android:** Funciona perfeitamente no Google Chrome. O navegador pedirá permissão e lerá a NTAG213.
- **iOS (iPhone):** A Apple **bloqueia** o uso do leitor de NFC pelo navegador (Safari ou Chrome no iOS).
- **Solução para a sala de aula:** Como os grupos terão vários alunos, basta garantir que o "Scanner Oficial" da equipe use um celular Android.

A Arquitetura do Rali (Passo a Passo)

Passo 1: O Banco de Dados (Google Sheets)

Crie uma planilha no seu Google Drive com duas abas:

1. **Aba `Missoes`**: Colunas `Tag_ID` | `Pergunta` | `Resposta_Curta` | `XP`
2. **Aba `Equipes`**: Colunas `Nome_Equipe` | `XP_Total`

Passo 2: A API Backend (Google Apps Script)

No Google Sheets, vá em **Extensões > Apps Script**. Você vai colar um código que recebe a resposta do aluno enviada pelo GitHub Pages, confere com a aba `Missoes`, soma o XP na aba `Equipes` e devolve um JSON de sucesso ou erro.

```
// Exemplo do backend no Apps Script (API POST)
function doPost(e) {
  var sheetMissoes = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Missoes");
  var sheetEquipes = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Equipes");

  var dados = JSON.parse(e.postData.contents);
  var tagLida = dados.tag_id;
  var respostaAluno = dados.resposta.toString().toLowerCase().trim();
  var nomeEquipe = dados.equipe;

  // Lógica de busca e validação (simplificada)
  var missoes = sheetMissoes.getDataRange().getValues();
  for (var i = 1; i < missoes.length; i++) {
    if (missoes[i][0] == tagLida) {
      if (missoes[i][2].toString().toLowerCase().trim() == respostaAluno) {
        // Resposta correta: Adicionar XP
        adicionarXP(sheetEquipes, nomeEquipe, missoes[i][3]);
        return ContentService.createTextOutput(JSON.stringify({"status": "sucesso", "xp_ganho": missoes[i]
          .setMimeType(ContentService.MimeType.JSON);
      } else {
        return ContentService.createTextOutput(JSON.stringify({"status": "erro", "mensagem": "Resposta Inc
          .setMimeType(ContentService.MimeType.JSON);
      }
    }
  }
}
```

Após colar o código, você clica em "Implantar > Nova implantação > App da Web", define o acesso para "Qualquer pessoa" e ele te dará uma ***URL da API****.*

Passo 3: O Aplicativo (GitHub Pages)

No GitHub, você criará um repositório simples com um arquivo `index.html`. Ele usará JavaScript puro para acessar o sensor NFC do celular Android e fazer um `fetch` (POST) para a URL do seu Apps Script.

```
<!-- Trecho da lógica de leitura NFC em HTML/JS -->
<button id="scanButton">Escanear Tag NFC</button>
<p id="mensagem"></p>

<script>
  document.getElementById('scanButton').addEventListener('click', async () => {
    try {
      const ndef = new NDEFReader();
      await ndef.scan();
      document.getElementById('mensagem').innerText = "Aproxime o celular da Tag...";

      ndef.onreading = event => {
        const decoder = new TextDecoder();
        for (const record of event.message.records) {
          const payloadId = decoder.decode(record.data);
          // O payloadId será 'tag_01'. Agora o JS libera o campo para o aluno
          // digitar a resposta e enviar para a URL do seu Google Apps Script!
          document.getElementById('mensagem').innerText = "Missão encontrada: " + payloadId;
        }
      };
    } catch (error) {
      document.getElementById('mensagem').innerText = "Erro ou NFC não suportado: " + error;
    }
  });
</script>
```

User prompt: Crie a planilha base do Google Sheets com as abas e missões de exemplo para eu começar.

Response: A planilha base do seu aplicativo já está criada no seu Google Drive, pronta para atuar como o banco de dados do sistema gamificado:

Rali SQL - Banco de Dados NFC

[file-tag: drive-uploaded-file-1HcitQ4K2-CQ0re8IiGIakfIK5JqjhMjhu08qT2IstGM]

Nesta planilha, deixei configuradas as duas abas fundamentais para a dinâmica de 100 minutos:

- **Missoes:** Contém as 5 perguntas de exemplo (compostas por comandos de DDL, DML e DQL), com o `Tag_ID` da sua NTAG213, a resposta curta esperada e a recompensa em XP.
- **Equipes:** Onde o App Script vai injetar a pontuação, já preenchida com as Equipes A e B zeradas.

Para fazer essa planilha se comunicar com o aplicativo web que ficará no GitHub Pages, você precisa transformá-la em uma API através do Google Apps Script.

1. **Abra o Google Apps Script**

   Criando o Servidor (Backend)

   Com a planilha aberta, clique no menu superior em **Extensões > Apps Script**. Um novo editor de código será aberto no seu navegador.

2. **Cole o código do Servidor**

   Salvando o Backend

   Apague qualquer código que estiver na tela e cole o script abaixo, depois clique no ícone de "Salvar" (disquete) ou pressione Ctrl+S.

3. **Gere a URL de acesso público**

   Deploy da API

   No canto superior direito, clique em **Implantar > Nova implantação**.

   - Em *Selecione o tipo*, clique na engrenagem e escolha **App da Web**.
   - Em *Quem pode acessar*, altere para **Qualquer pessoa**.
   - Clique em **Implantar** e autorize o acesso à sua conta do Google (ele pedirá permissão na primeira vez).

Aqui está o código completo para o **Apps Script** (Passo 2). Ele vai buscar a aba "Missoes", validar se a resposta que o aluno digitou bate com a resposta curta salva e, se acertar, vai na aba "Equipes" para somar o XP à equipe.

```
function doPost(e) {
  // Abre as duas abas da sua planilha
  var sheetMissoes = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Missoes");
  var sheetEquipes = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Equipes");

  // Recebe e formata os dados enviados pelo GitHub Pages
  var dados = JSON.parse(e.postData.contents);
  var tagLida = dados.tag_id; // Ex: 'tag_01'
  var respostaAluno = dados.resposta.toString().toLowerCase().trim();
  var nomeEquipe = dados.equipe; // Ex: 'Equipe A'

  // Busca os dados das missões
  var missoes = sheetMissoes.getDataRange().getValues();

  // Percorre as missões para encontrar a tag correspondente
  for (var i = 1; i < missoes.length; i++) {
    if (missoes[i][0] == tagLida) {

      var respostaGabarito = missoes[i][2].toString().toLowerCase().trim();

      if (respostaGabarito == respostaAluno) {
        var xpGanho = parseInt(missoes[i][3]);

        // Função interna para adicionar o XP
        adicionarXP(sheetEquipes, nomeEquipe, xpGanho);

        // Retorna o sucesso para o aplicativo web
        return ContentService.createTextOutput(JSON.stringify({
          "status": "sucesso",
          "mensagem": "Correto! +" + xpGanho + " XP",
          "xp_ganho": xpGanho
        })).setMimeType(ContentService.MimeType.JSON);
      } else {
        // Retorna o erro se a resposta não bater
        return ContentService.createTextOutput(JSON.stringify({
          "status": "erro",
          "mensagem": "Resposta Incorreta. Tente novamente!"
        })).setMimeType(ContentService.MimeType.JSON);
      }
    }
  }

  // Caso escaneie uma tag que não está na planilha
```

```
  return ContentService.createTextOutput(JSON.stringify({
    "status": "erro",
    "mensagem": "Tag não encontrada no banco de dados."
  })).setMimeType(ContentService.MimeType.JSON);
}

// Função para procurar a equipe na aba e somar a pontuação
function adicionarXP(sheetEquipes, nomeEquipe, xp) {
  var equipes = sheetEquipes.getDataRange().getValues();
  for (var j = 1; j < equipes.length; j++) {
    if (equipes[j][0] == nomeEquipe) {
      var xpAtual = parseInt(equipes[j][1]);
      sheetEquipes.getRange(j + 1, 2).setValue(xpAtual + xp);
      return;
    }
  }
}
```

Quando terminar a implantação, você receberá um **URL do App da Web** (vai começar com `[https://script.google.com/macros/s/](https://script.google.com/macros/s/)...`). Guarde essa URL! Nós vamos colocá-la dentro do seu arquivo `index.html` para ele saber com qual planilha deve se comunicar.

User prompt: Essas foram as atividades em sala tem como transformar em missões para atividade?

Response: Com base no roteiro da academia do arquivo `ATIVIDADE_CASA_ACADEMIA_07112026.pdf`, podemos adaptar perfeitamente a atividade de modelagem para a dinâmica de Rali com validação de **resposta curta** no seu aplicativo.

Como essa atividade envolve desde o modelo conceitual até o físico, os desafios testam se o aluno compreendeu a lógica da transição entre os modelos e a sintaxe do SQLite.

Aqui está a estrutura de missões prontas para você copiar e colar na aba **Missoes** da sua planilha no Google Sheets:

Missões: Rali da Academia (Modelagem e Físico)

Tag_ID	Pergunta (Para aparecer no App)	Resposta_Curta (Gabarito no Sheets)	XP
`tag_acad_01`	**Conceitual:** Na relação ALUNO (1,1) contrata PLANO (0,N), qual tabela deve receber a Chave Estrangeira (FK) no modelo lógico?	`aluno`	10
`tag_acad_02`	**Lógico:** Qual é o nome exato do atributo identificador (Chave Primária) da entidade PLANO?	`cod_plano`	10
`tag_acad_03`	**Físico (Tipagem):** Qual foi o tipo de dado sugerido no roteiro para armazenar o campo `valor_mensal`? (Apenas o nome do tipo, sem números).	`decimal`	20
`tag_acad_04`	**Caça ao Bug:** Qual é o comando SQL gerado automaticamente pelo brModelo que precisamos APAGAR porque o SQLite não aceita?	`alter table`	30
`tag_acad_05`	**Configuração:** Qual é o comando completo usado no início do script para o motor do SQLite ligar a verificação de Chaves Estrangeiras?	`pragma foreign_keys = on`	40
`tag_acad_06`	**Boss (Desafio Lógico):** Se a cardinalidade do cenário fosse (0,N) nos dois lados, o que surgiria obrigatoriamente no modelo lógico?	`nova tabela`	50

Nota sobre a validação: Como a sua API no Apps Script já converte a resposta do aluno para minúsculas e remove espaços sobressalentes (`toLowerCase().trim()`), os alunos não serão penalizados por digitar "Aluno" ou "ALUNO".

Como rodar esta dinâmica no Laboratório

1. **Distribuição Física:** Grave essas 6 missões nas tags NTAG213. Cole as tags `01` e `02` (Conceitual/Lógico) no quadro branco, a `03` e `04` (Físico/Erros do brModelo) nos monitores das bancadas laterais, e deixe a `05` e `06` (Boss) na sua mesa.
2. **Execução:** O aluno escaneia a tag, volta para a bancada para conferir o diagrama conceitual ou o script exportado do brModelo no PC, descobre a resposta (ex: descobre que o erro do brModelo está no `ALTER TABLE`) e digita no app para ganhar o XP.

Se quiser extrair desafios similares dos outros roteiros (como a Biblioteca em SQL ou as 16 Questões CRUD), posso gerar as tabelas de missões padronizadas para eles também.

User prompt: Quero

Response: Parece que a sua mensagem foi enviada antes de você terminar de digitar.

Você quer que eu gere as tabelas de missões para as outras atividades em PDF (como a da Biblioteca ou as questões de CRUD)? Ou prefere avançar para a criação do código do aplicativo (o arquivo `index.html`) para rodar no GitHub Pages?

Me diga qual é o próximo passo que você quer dar!

User prompt: Se quiser extrair desafios similares dos outros roteiros (como a Biblioteca em SQL ou as 16 Questões CRUD), posso gerar as tabelas de missões padronizadas para eles também. Quero

Response: Maravilha! Vamos transformar os seus roteiros de prática em três pacotes de missões gamificadas. Como o seu backend do Apps Script vai transformar tudo o que o aluno digitar em letras minúsculas e tirar os espaços sobrando, as "Respostas Curtas" aqui já estão otimizadas para evitar que o aluno perca pontos por bobagem de formatação.

Aqui estão as tabelas prontas para você copiar e colar na sua planilha do Google Sheets, separadas por tema:

Pacote 1: Rali dos 16 Comandos CRUD (Laboratório Prático)

Baseado no arquivo `SQL_Online_16_Questoes_CRUD_Aluno.pdf`, onde o foco é agilidade de código DML e DQL.

Tag_ID	Pergunta (Para aparecer no App)	Resposta_Curta (Gabarito)	XP
`tag_crud_01`	**CREATE:** Ao inserir um bem informando apenas descrição e setor, o que o banco insere automaticamente nas colunas vazias?	`null`	10
`tag_crud_02`	**READ:** Qual função agregadora usamos junto com GROUP BY para descobrir a quantidade total de bens em uma situação?	`count`	10
`tag_crud_03`	**READ/SORT:** Qual cláusula usamos no final do SELECT para ordenar os bens alfabeticamente ou do mais caro para o mais barato?	`order by`	20
`tag_crud_04`	**UPDATE:** Qual cláusula é de uso obrigatório no UPDATE para impedir que ele altere o setor e a situação da tabela inteira por acidente?	`where`	30
`tag_crud_05`	**DELETE (Armadilha):** Se o aluno executar a questão 16 `DELETE FROM bens;` sem filtro, o que acontecerá com os dados da tabela?	`apaga tudo`	50

Pacote 2: Rali da Biblioteca (Foco em Relacionamentos e JOIN)

Baseado no arquivo `Biblioteca_em_SQL_CRUD_JOIN.pdf`, perfeito para consolidar a transição entre múltiplas tabelas.

Tag_ID	Pergunta (Para aparecer no App)	Resposta_Curta (Gabarito)	XP
`tag_bib_01`	**Chaves:** Qual restrição aplicamos no `id` para garantir que ele não se repita, sendo o identificador único do aluno?	`primary key`	10
`tag_bib_02`	**Integridade:** Ao tentar dar DELETE em um aluno, o banco dá erro pois ele ainda possui um empréstimo. Isso é a defesa da chave...	`estrangeira`	20
`tag_bib_03`	**INNER JOIN:** Que cláusula escrevemos logo após o JOIN para ensinar ao banco que a FK de uma tabela é igual à PK da outra?	`on`	30
`tag_bib_04`	**LEFT JOIN:** Se usarmos LEFT JOIN, os alunos que nunca pegaram livros aparecerão com qual valor nas colunas de empréstimo?	`null`	40
`tag_bib_05`	**Apelidos:** O uso de letras como `a`, `e` e `l` logo após o nome da tabela no `FROM` serve para criar qual recurso na consulta?	`apelido`	20

Pacote 3: Rali do Supermercado (O "Boss" de Verdadeiro ou Falso)

Baseado no arquivo `Supermercado_em_SQL_CRUD_JOINs_Alunos...pdf`. Como a atividade original é de V ou F, os alunos devem rodar a prova real no SQLite Online e responder "verdadeiro" ou "falso" no app.

Tag_ID	Pergunta (Para aparecer no App)	Resposta_Curta (Gabarito)	XP
`tag_sup_01`	**Teste 01:** No SQLite, escrever a palavra apenas `JOIN` produz exatamente o mesmo resultado que `INNER JOIN`. (Verdadeiro ou Falso)	`verdadeiro`	10
`tag_sup_02`	**Teste 13:** Em SQL, para testar se a categoria do produto está vazia, a forma correta é usar `= NULL`. (Verdadeiro ou Falso)	`falso`	10
`tag_sup_03`	**Teste 03:** O cruzamento total das tabelas pelo `CROSS JOIN` exige que exista uma cláusula `ON` para funcionar. (Verdadeiro ou Falso)	`falso`	20
`tag_sup_04`	**Teste 27:** Uma Foreign Key (chave estrangeira) nunca pode apontar para uma coluna que não seja Primary Key. (Verdadeiro ou Falso)	`falso`	30
`tag_sup_05`	**Versão:** Qual função do sistema usamos com um `SELECT` para descobrir se o SQLite suporta `RIGHT JOIN` e `FULL JOIN` nativamente?	`sqlite_version`	50

💡 Dica para a sua Aula de 100 Minutos

Como você já desenhou o roteiro físico perfeito para a aula e tem esses três pacotes: Você pode fazer uma **Jornada Progressiva** no semestre.

- No primeiro sábado letivo, usa as tags da *Academia* e do *CRUD* juntas.
- No próximo, atualiza a planilha e o laboratório com as tags da *Biblioteca* e do *Supermercado*. Os alunos usam o mesmo app sem você precisar programar tudo de novo.

Se você já colou o código no Apps Script e pegou a **URL do App da Web** (que fizemos no passo anterior), nós já podemos escrever o arquivo `index.html` (o frontend) para você jogar no GitHub Pages e ver a mágica acontecendo no celular! Quer que eu crie o código do frontend agora?

User prompt: Sim quero

Response: Aqui está o código completo do seu aplicativo de leitura NFC. Como combinamos, criei tudo em um único arquivo `index.html`, que já inclui o visual moderno (usando Tailwind CSS), a lógica de leitura da etiqueta e a conexão com a sua planilha.

Vou gerar o arquivo agora. Nele, você só precisará colar aquela **URL do App da Web** (que você gerou no Google Apps Script) na linha 79 do código.

Próximos Passos (Para colocar no ar em 5 minutos):

1. **Substitua a URL:** Edite o arquivo HTML, vá até a linha 79 e troque o texto `"COLE_AQUI_A_SUA_URL_DO_APP_DA_WEB"` pela URL real que o Google gerou para você (entre aspas).
2. **Hospede no GitHub Pages:**
    - Crie um novo repositório público no seu GitHub (ex: `rali-sql`).
    - Faça o upload (Add file > Upload files) deste `index.html` para a pasta principal.
    - No repositório, vá em **Settings** (Configurações) > **Pages**.
    - Em "Source", selecione a branch `main` e salve.
    - Em cerca de 1 ou 2 minutos, o GitHub vai te dar o link oficial (ex: `[https://seu-usuario.github.io/rali-sql](https://seu-usuario.github.io/rali-sql)`).

Pronto! É só pedir para o líder de cada grupo acessar esse link pelo Google Chrome no Android e encostar nas tags que você colou no laboratório.

Quer alguma ajuda extra com a formatação das tags no aplicativo *NFC Tools* ou está tudo pronto para os testes?

User prompt: Ao total tem quantas missões?

Response: Ao todo, nós estruturamos **26 missões** prontas para você colocar nas suas tags NFC!

Aqui está a divisão exata do que criamos a partir dos seus materiais:

- **5 missões** do Rali MVP inicial (já cadastradas na aba "Missoes" da sua planilha).
- **6 missões** da Academia (focadas em Modelagem, DDL e Físico).
- **5 missões** do Rali CRUD (operações básicas de DML e DQL).
- **5 missões** da Biblioteca (focadas em relacionamentos e JOINs).
- **5 missões** do Supermercado (desafios de Verdadeiro ou Falso de nível Boss).

Como você tem **100 etiquetas NTAG213**, você pode gravar todas essas 26 missões de uma vez em etiquetas diferentes e apenas ir trocando quais você espalha pelo laboratório dependendo do tema da aula daquele sábado!

Quer que eu consolide essas 21 novas missões em um formato rápido para você copiar e colar de uma só vez na sua planilha do Google Sheets?

User prompt: Feche 30 questões

Response: Para fecharmos exatamente **30 missões**, vou adicionar 4 desafios extras focados nos subconjuntos de Transação (DTL) e Controle (DCL) que você abordou no roteiro "Explorando os Subconjuntos do SQL", além de um detalhe clássico de sintaxe.

Aqui estão as 4 missões finais para você colar na sua planilha e completar a sua coleção:

Pacote 4: Fechamento (DTL, DCL e Sintaxe)

Tag_ID	Pergunta (Para aparecer no App)	Resposta_Curta (Gabarito)	XP
`tag_extra_01`	**DTL (Salvar):** Qual é o comando utilizado para confirmar e salvar definitivamente as alterações de uma transação no banco de dados?	`commit`	20
`tag_extra_02`	**DTL (Desfazer):** Se você inseriu um dado errado dentro de uma transação iniciada com `BEGIN`, qual comando você usa para cancelar e desfazer a ação?	`rollback`	20
`tag_extra_03`	**DCL (Permissão):** Qual comando DCL é utilizado de forma teórica para dar permissão de leitura (`SELECT`) a um usuário estudante?	`grant`	30
`tag_extra_04`	**Sintaxe (Nulos):** A expressão `= NULL` nunca é verdadeira. Qual é a forma correta em SQL para testar a ausência de valor?	`is null`	30

Resumo do seu Arsenal de 30 Missões:

1. **MVP Base:** 5 missões (Mistura de DDL, DML e DQL).
2. **Academia:** 6 missões (Modelagem Lógica/Física).
3. **16 Comandos:** 5 missões (Agilidade no CRUD).
4. **Biblioteca:** 5 missões (Chaves e Relacionamentos JOIN).
5. **Supermercado:** 5 missões (Boss: Verdadeiro ou Falso).
6. **Fechamento:** 4 missões (Transações e Controle). **Total: 30 Missões!**

Se quiser, posso chamar a ferramenta integrada do Google Sheets e inserir todas essas novas 25 missões automaticamente na aba "Missoes" daquela sua planilha `Rali SQL – Banco de Dados NFC`. Quer que eu faça essa inserção em massa para você economizar tempo?

User prompt: Quero

Response: Prontíssimo! As **25 novas missões** foram inseridas diretamente na aba "Missoes" da sua planilha. Você agora tem um total de 30 desafios cadastrados e formatados no banco de dados.

Você pode conferir a atualização clicando aqui: Rali SQL - Banco de Dados NFC

[file-tag: drive-uploaded-file-1HcitQ4K2-CQ0re8IiGIakfIK5JqjhMjhu08qT2IstGM]

O que você precisa fazer agora:

1. Grave essas tags novas usando o seu celular (IDs de `tag_acad_01` até `tag_extra_04`).
2. Coloque o aplicativo no ar pelo GitHub Pages com a **URL do seu App da Web**.
3. Reúna a turma de alunos para a caçada ao código.

Toda vez que você quiser rodar uma dinâmica diferente, basta colar a etiqueta correspondente na bancada, e o app puxará a missão correta baseada no tema da semana.

Se surgir qualquer dúvida na hora de testar a primeira leitura NFC com a planilha ao vivo, é só me chamar. Boa aula com o Rali SQL!
